

Core CS Subjects — Operating Systems

8 Important Comparison Questions • Deep Hinglish • Interview Preparation

Source image mein diye gaye OS topics — Processes vs Threads, Scheduling, Deadlock, Semaphores/Mutex, Paging/Segmentation aur Virtual Memory — ko base bana kar ye comparison tables prepare ki gayi hain.

Q1. Process vs Thread

Point	Process ek running program ka independent execution unit hai. Iska apna execution context aur address space hota hai.	Thread process ke andar execution ka lightweight unit hai. Ek process mein multiple threads ho sakte hain.
2. Memory	Process ka address space normally doosre process se isolated hota hai.	Same process ke threads code, data aur heap share karte hain; stack/register context alag hota hai.
3. Resources	Process resources ka major owner hota hai, jaise address space aur OS-managed resources.	Thread process ke resources ko share karta hai.
4. Creation cost	Process create karna comparatively heavy/expensive hota hai.	Thread create karna comparatively lightweight hota hai.

Point	Process ek running program ka independent execution unit hai. Iska apna execution context aur address space hota hai.	Thread process ke andar execution ka lightweight unit hai. Ek process mein multiple threads ho sakte hain.
5. Context switching	Process switch mein address-space aur execution context change hone ki wajah se overhead relatively zyada ho sakta hai.	Same process ke threads ke beech switch generally lighter hota hai.
6. Communication	Processes ko IPC jaise pipe, message passing ya shared memory ki zarurat ho sakti hai.	Threads shared memory use kar sakte hain, lekin shared data ko synchronize karna padta hai.
7. Failure impact	Ek process crash ho to normally doosra independent process directly crash nahi hota.	Ek thread ki serious failure same process ke other threads ko affect kar sakti hai.
8. Interview one-liner	Process = independent execution environment.	Thread = process ke andar lightweight execution path.

□ **Yaad rakho: Process heavy + isolated; Thread lightweight + shared memory.**

Q2. FCFS vs SJF

Point	First Come First Serve.	Shortest Job First.
2. Selection rule	Ready queue mein jo process pehle aata hai, usko pehle CPU milta hai.	Jiska CPU burst sabse chhota hai, usko preference milti hai.
3. Basic nature	Typical FCFS non-preemptive scheduling hai.	Basic SJF non-preemptive hota hai; preemptive version SRTF kehlata hai.
4. Waiting time	Long process front mein ho to short processes ko zyada wait karna pad sakta hai.	Known burst times ke case mein average waiting time minimize karne ke liye strong approach hai.
5. Main problem	Convoy effect: ek long process ke peeche multiple short processes wait karte hain.	Long processes starvation face kar sakte hain agar short jobs baar-baar aati rahen.
6. Information required	Arrival order main information hai.	CPU burst time ka estimate/knowledge chahiye.
7. Practical use	Simple batch-style scheduling samajhne ke liye easy model hai.	Short jobs ko jaldi complete karne ke liye useful concept hai.
8. Interview one-liner	FCFS = arrival-order based scheduling.	SJF = shortest-burst based scheduling.

□ **Trick: FCFS = arrival order, SJF = shortest burst.**

Q3. FCFS vs Round Robin

Point	Arrival order follow hota hai.	Ready queue ke processes ko turn-by-turn fixed time quantum milta hai.
2. Preemption	Typical FCFS non-preemptive hai.	Round Robin preemptive hai.
3. Time quantum	Time quantum nahi hota.	Time quantum RR ka central concept hai.
4. Fairness	Long process CPU pakad le to baaki processes ko wait karna padta hai.	Har ready process ko repeated turns milte hain, isliye fairness better hoti hai.
5. Response time	Interactive workload mein long first job response delay kar sakti hai.	Small/appropriate quantum se interactive response generally better hota hai.
6. Overhead	Context switches comparatively kam ho sakte hain.	Quantum bahut small ho to context-switch overhead badh sakta hai.
7. Quantum effect	Quantum choose karne ka issue nahi.	Quantum bahut large ho to RR FCFS jaisa behave kar sakta hai; bahut small ho to overhead badhta hai.
8. Interview one-liner	FCFS = first arrival gets CPU first.	RR = cyclic scheduling + fixed time quantum.

☐ **Trick: RR mein time quantum + preemption + fairness sabse important.**

Q4. SJF vs Priority Scheduling

Point	Shortest CPU burst wala process select hota hai.	Highest-priority process select hota hai.
2. Main goal	Average waiting time ko reduce/minimize karna.	Important/high-priority work ko preference dena.
3. Preemption	Basic SJF non-preemptive; SRTF preemptive variant hai.	Priority scheduling non-preemptive ya preemptive dono ho sakti hai.
4. Required information	CPU burst time ka estimate important hai.	Priority value/rank important hai.
5. Starvation	Long jobs starvation face kar sakti hain.	Low-priority jobs starvation face kar sakti hain.
6. Common solution	Scheduling policy ko long jobs ke wait ke liye carefully design karna hota hai.	Aging se waiting process ki priority gradually increase ki ja sakti hai.
7. Decision example	P1 burst 3 aur P2 burst 10 ho to SJF mein P1 prefer hoga.	P1 priority low aur P2 priority high ho to Priority scheduling mein P2 prefer hoga.
8. Interview one-liner	SJF = shortest burst first.	Priority = highest priority first.

☐ **Trick: SJF burst time dekhta hai; Priority priority value dekhta hai.**

Q5. Mutex vs Semaphore

Point	Mutex ek mutual-exclusion lock hai: ek time par critical resource ko normally ek owner thread hold karta hai.	Semaphore ek counter-based synchronization mechanism hai jo available permits/resources ko represent kar sakta hai.
2. Ownership	Mutex ka ownership concept hota hai; lock karne wala thread normally unlock karta hai.	Semaphore generally ownership lock nahi hota; wait/signal operations permits ko control karte hain.
3. Value	Mutex ko simple locked/unlocked state ke roop mein samjha ja sakta hai.	Semaphore ki value 0, 1, ya greater ho sakti hai depending on type/use.
4. Main use	Critical section ko protect karna.	Multiple identical resources ya signaling/coordination ko control karna.
5. Binary case	Mutex inherently mutual exclusion ke liye designed hai.	Binary semaphore ki value commonly 0/1 hoti hai, lekin semantics mutex se identical nahi hote.

Point	Mutex ek mutual-exclusion lock hai: ek time par critical resource ko normally ek owner thread hold karta hai.	Semaphore ek counter-based synchronization mechanism hai jo available permits/resources ko represent kar sakta hai.
6. Example	Shared counter update karte waqt mutex lock karke ek thread ko critical section mein allow karna.	3 identical resources available hon to counting semaphore initial value 3 rakh sakte hain.
7. Error/discipline	Incorrect unlock/lock discipline bugs create kar sakti hai.	Incorrect wait/signal operations deadlock ya resource accounting problems create kar sakte hain.
8. Interview one-liner	Mutex = ownership-based mutual exclusion.	Semaphore = permit/count based synchronization.

□ **Trick: Mutex = one owner/lock; Semaphore = count/permits.**

Q6. Deadlock vs Deadlock Prevention

Point	Deadlock mein processes ek doosre ke resources/events ka wait karte rahte hain aur progress ruk jati hai.	Deadlock prevention mein system design aisa hota hai ki deadlock ki necessary condition ko break kiya ja sake.
2. State	System already stuck state mein ho sakta hai.	Goal deadlock ko hone hi na dena hai.

Point	Deadlock mein processes ek doosre ke resources/events ka wait karte rahte hain aur progress ruk jati hai.	Deadlock prevention mein system design aisa hota hai ki deadlock ki necessary condition ko break kiya ja sake.
3. Conditions	Classic necessary conditions: mutual exclusion, hold and wait, no preemption, circular wait.	Prevention techniques in conditions mein se kam se kam ek ko deliberately break karti hain.
4. Hold and wait	Deadlock situation mein process kuch resources hold karke aur resources ka wait kar sakta hai.	Prevention mein process se saare required resources ek saath request karwane jaisi policy use ki ja sakti hai.
5. Circular wait	Processes circular dependency bana sakte hain.	Resources ko fixed ordering mein acquire karwana circular wait ko prevent kar sakta hai.
6. No preemption	Resource forcefully release nahi hota to waiting chain ban sakti hai.	Kuch systems mein resource preemption policy se deadlock risk reduce kiya ja sakta hai.
7. Trade-off	Deadlock ke time productivity/progress ruk sakti hai.	Prevention safe ho sakti hai but resource utilization ya concurrency reduce kar sakti hai.
8. Interview one-liner	Deadlock = processes permanently wait for each other.	Prevention = deadlock ki necessary condition ko break karo.

□ **Trick: Prevention deadlock ki necessary condition ko break karti hai.**

Q7. Paging vs Segmentation

Point	Paging memory ko fixed-size pages aur physical memory ko fixed-size frames mein divide karta hai.	Segmentation program ko logical variable-size segments jaise code, data, stack mein divide karta hai.
2. Size	Pages fixed-size hote hain.	Segments variable-size hote hain.
3. Programmer view	Paging mostly physical-memory management ke perspective se transparent hoti hai.	Segmentation program ke logical structure ko reflect karti hai.
4. Fragmentation	Paging mein internal fragmentation ho sakti hai.	Segmentation mein external fragmentation ho sakti hai.
5. Address	Logical address generally page number + offset hota hai.	Logical address generally segment number + offset hota hai.
6. Protection/sharing	Page-level protection/sharing possible hai.	Segment-level protection/sharing logical units ke saath naturally map ho sakti hai.
7. Allocation	Fixed-size frames ki wajah se external fragmentation avoid hoti hai.	Variable-size allocation ki wajah se free memory gaps create ho sakte hain.
8. Interview one-liner	Paging = fixed-size memory blocks.	Segmentation = variable-size logical program blocks.

☐ **Trick: Paging fixed-size; Segmentation variable-size logical units.**

Q8. Virtual Memory vs Physical Memory

Point	Virtual memory process ko large logical address space ka abstraction deti hai.	Physical memory actual RAM hai jahan currently resident data/instructions stored hote hain.
2. Location	Virtual address process ke address space ka part hai; backing storage ke saath mapping ho sakti hai.	Physical memory RAM hardware mein hoti hai.
3. Size	Virtual address space physical RAM se larger ho sakta hai.	Physical memory ki capacity actual installed RAM par depend karti hai.
4. Address translation	CPU/process virtual address generate kar sakta hai.	MMU/page tables virtual address ko physical address/frame se map karte hain.
5. Page fault	Required page RAM mein nahi hai to virtual-memory system page fault trigger kar sakta hai.	RAM mein page available nahi hone par OS ko storage se page load karna pad sakta hai.
6. Purpose	Isolation, large address space aur efficient memory use provide karna.	Running processes ko actual fast main memory provide karna.

Point	Virtual memory process ko large logical address space ka abstraction deti hai.	Physical memory actual RAM hai jahan currently resident data/instructions stored hote hain.
7. Speed	Virtual-memory access usually translation/cache mechanisms par depend karta hai; page fault bahut expensive ho sakta hai.	RAM access storage access se much faster hota hai.
8. Interview one-liner	Virtual memory = logical/abstract memory view.	Physical memory = actual RAM.

☐ **Trick: Virtual = logical view; Physical = actual RAM.**

☐ **8 Comparisons — Ultra-Short Revision**

1. Process vs Thread → isolated vs shared execution.
2. FCFS vs SJF → arrival order vs shortest burst.
3. FCFS vs RR → no quantum vs fixed quantum.
4. SJF vs Priority → burst length vs priority value.
5. Mutex vs Semaphore → ownership lock vs permits/count.
6. Deadlock vs Prevention → stuck state vs condition-breaking strategy.
7. Paging vs Segmentation → fixed-size vs variable-size logical blocks.
8. Virtual vs Physical Memory → logical address space vs actual RAM.